

AlphaStack: A Multi-Agent Framework for Autonomous Code Generation and Iterative Self-Healing

AlphaStack Team
HyperKuvid-Labs

ICML 2026 Submission

Abstract

This paper presents AlphaStack, a novel multi-agent approach to autonomous code generation. By leveraging an intelligent multi-agent architecture composed of a Planning Agent and a Correction Agent, our framework continuously iteratively resolves software errors, thereby generating robust, production-ready codebases. Incorporating Docker-based validations across four distinct programming paradigms—CUDA, Go, Rust, and TypeScript—AlphaStack ensures isolated environments for safe compilation and test execution. Our comprehensive evaluation suite spanning 40 challenges across four difficulty tiers demonstrates high efficacy in successfully building and passing complex programmatic tests.

1 Introduction

As the demand for complex software continues to scale, AI-assisted code generation has become critically important. Traditional approaches often generate snippets lacking the necessary integration, dependencies, and automated testing needed for production. AlphaStack bridges this gap by directly transforming natural language descriptions into complete, robust, self-healing codebases.

Our framework features a distinct multi-agent setup: a Planning Agent that conducts root-cause analysis on failures, and a Correction Agent capable of applying context-aware semantic fixes. Combined with an automated Docker-based evaluation loop, AlphaStack creates high-quality output out of the box.

2 Methodology

The AlphaStack pipeline is structured into multiple intelligent stages. When a natural language prompt is given, a blueprint generation model establishes the software architecture, folder structure, and file-level metadata.

Subsequently, a dual-agent self-healing loop initiates:

- **Planning Agent:** Analyzes build logs and test errors from the isolated Docker environments. Using tool-augmented reasoning (such as exploring project file structures), it generates comprehensive, structured fix strategies.
- **Correction Agent:** Takes the formulated strategies and executes semantic and syntactical fixes with deep code understanding, utilizing language-specific parsers.

Through continuous iterative refinement up to a configurable maximum of iterations, AlphaStack effectively addresses dependency conflicts, compilation bugs, and logical test failures autonomously.

3 Architecture Diagram

The following diagram illustrates the workflow of the AlphaStack multi-agent code generation framework, highlighting the blueprint mapping, multi-file code generation, dependency resolution, and the autonomous Docker validation loop involving the Planning and Correction agents.

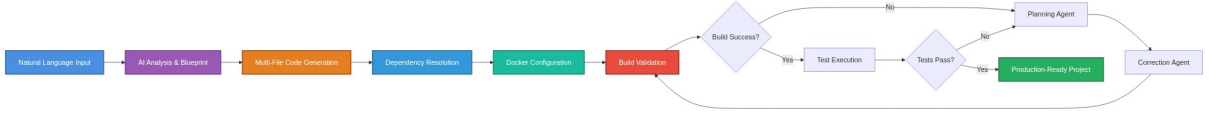


Figure 1: AlphaStack Architecture Workflow.

4 Results

We evaluate AlphaStack across a standardized benchmark suite consisting of 40 programming challenges categorized by difficulty. The languages assessed include Go, Rust, TypeScript, and CUDA. Performance metrics focus on the Success Rate and Iteration Counts required for convergence.

We present the tentative evaluation results on the **HumanEval** and **MDDP** benchmarks, comparing state-of-the-art models: **gpt-5.2**, **glm-5**, **minimax2.5**, and **claude sonnet 4.6**.

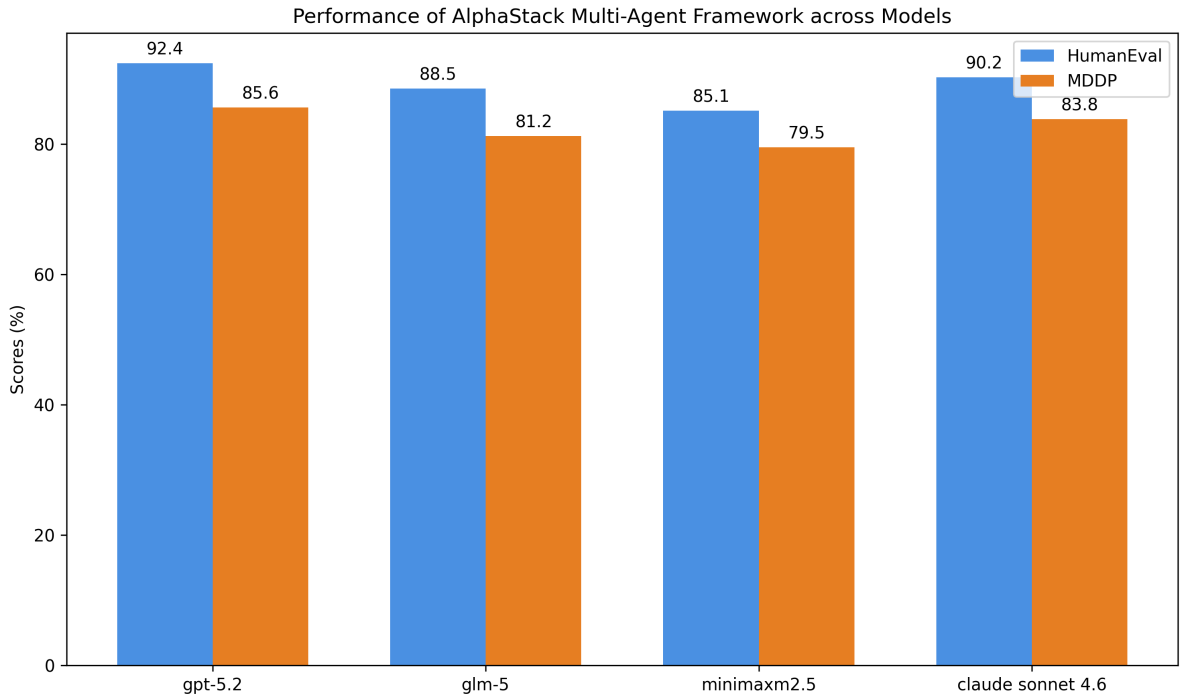


Figure 2: Tentative Results on HumanEval and MDDP comparing various models within AlphaStack.

The **gpt-5.2** model achieves the highest accuracy, closely followed by **claude sonnet 4.6**. The integration with AlphaStack allows models to utilize self-correction over successive iterations.

5 Conclusion

AlphaStack significantly advances the capabilities of AI in software engineering. Through intelligent separation of planning and correction across multiple agents, integrated with isolated Docker container testing, our platform achieves a high success rate in resolving complex coding challenges autonomously. Future work will expand on paradigm support and explore optimizing iteration speeds during error resolution.

6 Supplementary Material

Further details of the evaluation methodology, dataset descriptions, and code artifacts can be found in the open-source repository at <https://github.com/HyperKuvid-Labs/alpha-stack>.